

DevFlow — Complete Project Documentation for Backend Development

Purpose of this document: This is the single source of truth about the **DevFlow** product for anyone building its backend. It describes exactly what the product is, every feature it offers, how each feature behaves in the current frontend (so the backend can be built to match), the exact data model and business rules the frontend already implements, and a complete blueprint for implementing that backend with **Spring Boot (Java)**. It also lists the skills required. Length is intentional — every detail here was extracted from the actual frontend source code and is accurate.

Table of Contents

1. What DevFlow Is
 2. Current State of the Codebase
 3. Frontend Technology Stack
 4. Feature Inventory — Every Feature and How It Works
 5. Domain Model (Entities, Fields, Relationships)
 6. Business Rules & Invariants
 7. Roles & Permissions Matrix
 8. Realtime Requirements
 9. Proposed REST API Surface
 10. Proposed WebSocket Event Contracts
 11. Recommended Backend Architecture (Spring Boot)
 12. Database Schema (PostgreSQL DDL)
 13. Security Design
 14. Skills Required to Build This Backend
 15. Suggested Implementation Phases
 16. Testing Strategy
 17. Glossary
-

1. What DevFlow Is

DevFlow is a **multi-tenant, workspace-scoped project-management and issue-tracking web application** — in the same family as Linear, Jira, and Asana, but with a distinct philosophy: *"Built for workspaces, not boards alone."*

Its core mental model is:

- **Workspace** = the top-level tenant container (e.g. "Acme Labs"). All data is scoped to a workspace. A user can belong to many workspaces.
- **Project** = a work stream inside a workspace (e.g. "Frontend", "Backend", "DevOps"). Projects have their own member lists, their own issue-key prefix (`DEV-`, `API-`, `OPS-`), their own chat channel, and their own analytics.
- **Issue** = a ticket (feature, bug, task) that moves through a fixed **5-stage pipeline**: `Backlog` → `Todo` → `In Progress` → `Review` → `Done`. Issues carry priority, labels, an assignee, a reporter, threaded comments, linked git commits, and a full audit trail of status changes.
- **Everything is scoped**. Nothing is global except the workspace switcher. When you switch workspace or project, every page (dashboard, board, chat, members, analytics, AI assistant) re-scopes instantly.

The product also provides: **project-scoped realtime chat** (mentions, stickers, emoji reactions, typing indicators, code blocks), an **AI project assistant** (answers questions about status, team load, blockers, and next steps from live board data), **project-health analytics** (velocity, cycle time, status distribution), **invites** (Tinder-style accept/decline cards for projects and workspaces), a **notification center** (commit, mention, status, chat, and invite events), **global search** (🔍), **role-based member management**, **workspace/project settings**, a **personal account page** (avatar upload, presence, leave workspace), and a dual **theme system** (dark + soft-brutalist "brutal").

Tagline from the marketing page: "Every issue has a path." — every ticket carries a trace from creation to done: every status move, every commit, every comment, in order.

2. Current State of the Codebase

Critical context for the backend developer: DevFlow is currently a **100% frontend application with no real backend**.

- All application data lives in a single **Zustand store** (`src/store/useAppStore.js`) that is hydrated from hard-coded **seed data** (`src/data/seed.js`) and **persisted to the browser's** `localStorage` (key: `devflow-store`).
- "Sign in" is **mocked**: any valid email + password ≥ 4 chars signs you in as the user matching that email (defaults to user `u1`, "Ada Lovelace"). There are GitHub/Google buttons but they just call the same mock sign-in.
- "Realtime" chat, typing indicators, comment replies, and ambient commit activity are **simulated client-side** in `src/lib/realtime.js` with `setTimeout` — there is no socket server.
- There are **no network calls** anywhere in the app. The `code-searcher` confirmed there are zero `fetch` / `axios` /API calls; the only `localStorage` usage is theme + store persistence.

This means the Spring Boot backend must be built from scratch, and the frontend must be refactored later to call it. The REST API, WebSocket layer, database, auth, and authorization must be designed to reproduce the behaviors the frontend already demonstrates. This document defines those contracts precisely.

Frontend repository layout (for reference)

```
DevFlow/
├── index.html
├── package.json
├── vite.config.js
├── tailwind.config.js
├── src/
│   ├── main.jsx           # entry; theme engine
│   ├── App.jsx           # routing + auth guard
│   ├── index.css         # design tokens (dark/brutal themes)
│   ├── data/seed.js      # ALL seed/demo data (users, workspaces, projects, issues,
│   │   chat, reactions, invites, notifications)
│   ├── lib/
│   │   ├── constants.js  # STATUSES, PRIORITIES, LABELS, ROLES, navItems, colors
│   │   ├── utils.js      # uid, initials, randomSha, timeAgo, fmt...
│   │   ├── realtime.js   # simulated realtime layer (mock peers)
│   │   └── stickers.js   # sticker pack definitions
│   ├── store/useAppStore.js # THE entire "backend" today (CRUD + business rules +
│   │   persistence)
│   ├── pages/            # Landing, Auth, Dashboard, Board (via component), Chat,
│   │   Members, Analytics, AI, Settings, Account
│   └── components/
│       ├── kanban/       # Board (dnd-kit), NewIssueSlideOver
│       ├── issue/        # IssueDetailPage (slide-over with timeline/comments/commits)
│       ├── layout/       # AppShell, Sidebar, TopBar, SearchModal
│       ├── notifications/ # NotificationsPanel
│       ├── invites/      # InviteStack (Tinder cards)
│       ├── projects/     # CreateProjectModal
│       └── ui/           # design-system components (Button, Modal, Card, Avatar,
│   │   Dropdown...)
│       ├── landing/      # MiniKanban, PipelineRail
│       └── brutal/       # BrutalHero, BrutalMarquee, BrutalFeatureCard
```

3. Frontend Technology Stack

Understanding the frontend stack matters because the backend's API shapes and realtime contract must fit it.

Layer	Technology	Notes
Framework	React 18 (with <code>react-dom</code>)	Hooks + function components only
Build tool	Vite 5	Dev server, HMR, production build
Language	JavaScript (ESM)	<code>"type": "module"</code>
Routing	react-router-dom 6	<code>BrowserRouter</code> , route guards via <code>RequireAuth</code>
State	Zustand 4 + <code>persist</code> middleware	Single global store; <code>localStorage</code> persistence
Styling	Tailwind CSS 3 + custom CSS variables	Dark theme + "brutal" (soft-brutalist) theme; tokens like <code>--signal-teal</code> , <code>--surface-card</code> , <code>--text-ink</code>
Animations	framer-motion 11, GSAP 3 (ScrollTrigger/ScrollToPlugin)	Micro-interactions, page transitions, landing scroll story

Layer	Technology	Notes
Drag & drop	@dnd-kit (core, sortable, utilities)	Kanban board
Charts	recharts 2	Area/line/bar/pie charts for analytics
Dates	date-fns 4	Relative time, formatting
Icons	lucide-react	Icon set

Key architectural fact: The Zustand store (`useAppStore.js`) is effectively the backend today. Every action (create/update/delete, membership, invites, notifications, auth) is implemented there as a pure client-side state mutation. A faithful backend reproduces these semantics server-side.

4. Feature Inventory — Every Feature and How It Works

This section is organized per feature. For each feature you get: **what it does**, **how the frontend implements it today**, **the data it touches**, and **what the backend must provide**. Where relevant, exact constants and thresholds from the code are included so the backend matches behavior precisely.

4.1 Authentication & Account

What it does

- **Register:** full name + email + password (password ≥ 4 chars, email validated with regex `^\S+@\S+\.\S+$`). Creates an account.
- **Login:** email + password. Any email present in the seed users logs you in as that user; otherwise defaults to user `u1` (Ada Lovelace).
- **SSO buttons:** "Continue with GitHub" and "Continue with Google" (currently mocked — they just sign in the same way).
- **Sign out:** clears session, redirects to `/login`.
- **Route guard:** routes under `/app/*` redirect to `/login` when not signed in (`RequireAuth` in `App.jsx` checks `signedIn && currentUserId`).

How it works today (frontend)

```
// store/useAppStore.js
signIn: (email) => {
  const match = s.users.find(u => u.email.toLowerCase() === email.trim().toLowerCase());
  return { signedIn: true, currentUserId: match?.id ?? 'u1' };
},
signOut: () => set({ signedIn: false, currentUserId: null }),
```

The session is persisted to `localStorage` (the `persist` middleware keeps `signedIn` and `currentUserId`). On app boot, `AppShell` bootstraps a scoped context: if the user has no `activeWorkspaceId` , it selects the first workspace they belong to, or auto-creates "My Workspace" if they belong to none.

Account page (`/app/account`) lets a user edit:

- Profile photo — **upload** (file picker, image/*, max 2 MB, read as base64 data URL) **or paste an image URL**; replace the initials-avatar everywhere.
- Display name (max 40 chars, must be non-empty).
- Email (used for sign-in and mentions; max 80 chars).
- Avatar color (from a palette of 9 colors).
- Online presence toggle (drives the green "online" dot on avatars).
- **Leave workspace** (with confirmation modal; if you're the Owner, ownership transfers to the next member — see business rules).

Backend must provide

- `POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/refresh`, `POST /api/auth/logout`.
 - OAuth2 login with GitHub and Google (Spring Security OAuth2 client) — or at least the ability to plug it in.
 - JWT access + refresh token flow (frontend will need `Authorization: Bearer <token>` on every request).
 - User profile CRUD (`GET/PUT /api/users/me`), avatar upload endpoint (or presigned URL / object storage), presence flag update.
 - Password hashing (BCrypt/Argon2), email uniqueness enforcement.
 - A user object the frontend can render: `{ id, name, email, color, online, avatar? }`.
-

4.2 Workspaces (Multi-Tenancy)

What it does

- Workspaces are the top-level tenant container: name, **slug** (auto-derived lowercase from name: `name.toLowerCase().replace(/[^a-z0-9]+/g, '-')`), an **icon/logo** (from a set: `hexagon`, `layers`, `wave`, ...), an **accent color** (from `PROJECT_ACCENTS`), a description, and a **member list with roles** (`Owner` | `Admin` | `Member`).
- The sidebar has a **workspace switcher** dropdown; the top bar shows `Workspace / Project` breadcrumbs with switchers.
- **Create workspace** (from switcher or auto-created on first login). The creator becomes `Owner` and is the only member.
- **Delete workspace** (Owner only, danger zone in Settings) — cascades: deletes the workspace, its projects, their issues, their chat, and related notifications.
- **Leave workspace** (Account page).
- Workspaces a user is **not** a member of never appear in the switcher (confirmed by a seed workspace `ws3` that the demo user deliberately is not part of).

Data shape (seed)

```
{
  id: 'ws1', name: 'Acme Labs', slug: 'acme-labs', icon: 'hexagon',
  accent: '#009E88', description: '...',
  memberIds: ['u1', 'u2', 'u3', 'u4', 'u5', 'u6'],
  roles: { u1: 'Owner', u2: 'Admin', u3: 'Admin', u4: 'Member', u5: 'Member', u6: 'Member' },
}
```

Backend must provide

- `GET /api/workspaces` (only ones the caller is a member of), `POST /api/workspaces`, `GET/PUT/PATCH /api/workspaces/{id}`, `DELETE /api/workspaces/{id}` (Owner only, cascade).
- `GET /api/workspaces/{id}/members`, `POST /api/workspaces/{id}/members`, `PATCH /api/workspaces/{id}/members/{userId}/role`, `DELETE /api/workspaces/{id}/members/{userId}` (Owner/Admin only).
- `POST /api/workspaces/{id}/leave`.
- Slug generation + uniqueness constraint (in the seed, slugs are `acme-labs`, `vertex`, `pelican`).
- Every subsequent endpoint must enforce "caller must be a member of the workspace" (and "caller must be a member of the project" for project-scoped endpoints) — this is the **scoping invariant**.

4.3 Projects

What it does

- Projects live inside a workspace with their own member list and roles (`Owner` | `Member`).
- **Issue-key prefix** is auto-derived from the name: strip non-letters, take first 3 uppercase letters, fallback `'PRJ'` (e.g. "Frontend" → `DEV`, "Backend" → `API`, "DevOps" → `OPS`).
- Projects store integration metadata:
 - `repoUrl` — GitHub repository URL ("Open Repo" button on the board opens it).
 - `localRepoPath` — local clone path on the developer's machine.
 - `ideUrl` — optional IDE deep link override (e.g. `vscode://file/C:/...`); if absent, the board derives `vscode://file/<localRepoPath>` automatically.
- Project creation modal: name, description, repo URL, local path, IDE link, accent color.
- Deleting a workspace deletes its projects (there is no standalone "delete project" action in the current UI, but the backend should support it for parity).
- A workspace with **zero projects** shows an empty state with a "Create project" call-to-action on the board page.

Member-management rules (implemented in the store — replicate exactly):

- `addProjectMember(projectId, userId)` **refuses** if the user is not already a workspace member: toast "Failed to add member — user must be invited to the workspace first". (i.e., project membership is a subset of workspace membership.)
- `removeProjectMember` removes from `memberIds` and clears the `roles` entry.
- Adding a member grants role `Member` (project roles are only `Owner` / `Member`).

Backend must provide

- `GET /api/workspaces/{id}/projects` , `POST /api/workspaces/{id}/projects` , `GET/PUT/PATCH/DELETE /api/projects/{id}` .
- `POST /api/projects/{id}/members` , `DELETE /api/projects/{id}/members/{userId}` , `PATCH /api/projects/{id}/members/{userId}/role` .
- **Enforce the workspace-membership prerequisite** before adding a project member.
- Issue-key prefix derivation + uniqueness of prefixes within a workspace (nice-to-have; the frontend does not enforce uniqueness today).

4.4 Issues & the 5-Stage Pipeline

What it does — the heart of the product

Every issue is a ticket that travels a fixed pipeline of **5 statuses**:

Status	Label	Signal color (design tokens)
backlog	Backlog	muted
todo	Todo	azure
in_progress	In Progress	amber
review	Review	violet
done	Done	teal

An issue has:

- **Key** — human identifier, project-scoped and sequential: `{PREFIX}-{NNN}` zero-padded to 3 digits (`DEV-099` , `API-201` , ...). Generated by scanning the project's existing issues: `max(existing numbers) + 1` .
Not a database auto-increment across the whole table — it is per-project.
- **Title** (required) and **description** (Markdown: `**bold**` , ``code`` , `fences` , `- lists`).
- **Priority**: `high | medium | low` (coral / amber / teal accents). Cards get a 3px left border in the priority color.
- **Labels**: fixed vocabulary from `LABELS = [ 'bug', 'feature', 'design', 'perf', 'refactor', 'infra', 'ai', 'docs' ]` . Multi-select chips.
- **Assignee** (nullable — "Unassigned") and **Reporter** (the creating user).
- **Comments** — top-level comments plus **threaded replies** (1 level deep).
- **Linked commits** — `{ sha (40 hex), message, branch, authorId, createdAt }` . "Link commit" lets a user paste SHA/message/branch. SHAs render shortened (first 7 chars).
- **statusHistory** — an audit trail of every status move: `{ from, to, by (userId), at (ISO timestamp) }` . This powers the activity timeline, cycle-time analytics, and "who moved what" on the dashboard.
- **aiGenerated** flag — issues created by/with AI get a violet "AI" badge.
- **createdAt / updatedAt** timestamps (ISO strings).

How it works today (frontend)

- **Create**: "New Issue" button or per-column `+` button opens the `NewIssueSlideOver` (title, markdown description, status, priority, assignee, labels). Issue key is auto-assigned; `statusHistory` starts with `[{`

```
from: null, to: <status>, by: currentUser, at: now }]]
```

- **Update:** any field editable; title auto-saves with a **500 ms debounce**; description has an edit/preview mode with explicit Save.
- **Move:** via drag-and-drop on the board (records a statusHistory entry on every status change) or via a per-card "Move to" menu, or the status `<select>` in the issue detail slide-over.
- **Duplicate:** creates a copy with `(copy)` suffix in the title, resets status to `backlog`, clears comments/commits, fresh statusHistory.
- **Delete:** with confirmation modal ("This removes the issue and its history from the board. This can't be undone."), returns to the board.
- **Issue detail** is a **slide-over panel** (680px) rendered over a still-mounted board, reachable at `/app/issue/:issueId` (also from search). Escape or backdrop-click closes it.

Board behaviors (matching the frontend exactly):

- **WIP limits** (displayed, not enforced in the demo): `todo: 6`, `in_progress: 5`, `review: 4`; `backlog` / `done` unlimited. Column count badge shows `n/limit` and the progress bar turns coral when over limit.
- **Aging/stale detection:** an open issue with no update in **> 5 days** gets an amber "aging · Nd" chip + border; **> 8 days** turns coral.
- **Filters:** free-text (matches `key`, `title`, and `labels`), priority dropdown, assignee dropdown, "Clear filters".
- **Drag & drop:** pointer (6px activation distance) and keyboard sensors; reorder within a column and move between columns; a drag overlay ghost with a subtle rotate.
- **Column layout:** 5 fixed columns, each with header, count badge, WIP progress bar, and a `+` button; empty columns show "No issues — create your first to start".

Backend must provide

- `GET /api/projects/{id}/issues` (with filtering params: `status`, `priority`, `assigneeId`, `search`), `POST /api/projects/{id}/issues`, `GET /api/issues/{id}`, `PUT/PATCH /api/issues/{id}`, `DELETE /api/issues/{id}`, `POST /api/issues/{id}/duplicate`.
- `POST /api/issues/{id}/move` (or PATCH status) — **server-side must append to `statusHistory`** and emit a realtime event + notification.
- Key generation: per-project sequential key; must be **race-safe** (e.g. `SELECT MAX` inside the project's transaction, a per-project counter, or a unique index + retry).
- Comments: `POST /api/issues/{id}/comments`, `POST /api/comments/{id}/replies`.
- Commits: `POST /api/issues/{id}/commits` (link a commit).
- Read models for the timeline (status moves + commits + comments merged and sorted by timestamp).
- WIP limits and aging rules are **frontend-presentation concerns today**; the backend should still expose `updatedAt` and status reliably so the client can keep computing them. (Optionally enforce WIP server-side later.)

4.5 Kanban Board

Covered in depth above under "Board behaviors". Backend summary:

- Board = `GET /api/projects/{id}/issues` grouped by status client-side (or via `?groupBy=status`).
- Every mutation must be immediately consistent for all viewers → **realtime push** (WebSocket) on create/update/move/delete, because the board is a collaborative surface (see §8).
- Optional: WIP enforcement, `positions` field for stable column ordering when reordering within a column (the frontend does local reordering via `arrayMove` — if you want cross-client ordering persistence, add a `position` int per (issue, status) and expose a reorder endpoint).

4.6 Dashboard

What it does (`/app/dashboard`) — project-scoped live overview ("bento" grid):

- **Hero tile**: greeting by time of day ("Good morning/afternoon/evening"), project name, open-issue count, shipped-last-week count, a 14-day mini sparkline, "Open board" / "Ask the AI" CTAs.
- **Pipeline health**: work-by-stage horizontal bars per status with % done badge.
- **KPI tiles** (animated count-up): Open issues, In progress, Shipped · 7d, **Avg cycle time** (creation → done, in days).
- **Throughput velocity**: area chart, issues completed per day over the **last 14 days**, with a ▲/▼ week-over-week delta.
- **Status mix**: donut chart with center total.
- **Your issues**: up to 5 issues assigned to the current user, sorted by most recently updated.
- **Recent activity**: latest status moves (issue key → status label, actor avatar, relative time), up to 8 events.
- **Team load**: open issues per member with progress bars, plus an avatar stack.

Metrics formulas (implement client-side — replicate server-side or keep computing client-side from raw data):

- **Cycle time** (days) = $(\text{timestamp when status first became 'done'} - \text{createdAt}) / 86_400_000$, averaged over closed issues. "Done" timestamp comes from the last statusHistory entry with `to === 'done'`.
- **Throughput** = count of issues whose *done transition* fell within each of the last 14 calendar days.
- **Open** = `status !== 'done'`; **Done** = `status === 'done'`.
- **Stale** (used by AI assistant) = open issue with `updatedAt` older than 5 days.

Backend must provide

- Either a **project stats endpoint** (`GET /api/projects/{id}/stats` returning the aggregates) or, more simply and robustly, the raw issues + statusHistory so the client computes metrics as it does today.
Recommendation: provide both — raw list for the client's realtime-updated views, plus an aggregate endpoint for initial render.
- Activity feed: recent statusHistory entries with actor info (`GET /api/projects/{id}/activity?limit=8`).

4.7 Analytics ("Project Health")

What it does (`/app/analytics`) — a deeper, project-scoped analytics page:

- KPI callouts: Avg cycle time, Closed last 14d, Open issues, Total tracked.

- **Throughput velocity** line chart (14 days).
- **Distribution by status** bar chart.
- **Cycle time trend**: each closed issue plotted by its Done date (line), with a dashed reference line at the average.
- **Status split** donut + legend.
- **Cycle time card**: big number + commentary ("Pipeline velocity is healthy when this trends below 2.5 days."), plus demo-only "uptime 99.99%" and "commits linked" callouts.
- All charts are theme-aware (CSS variables).

Backend must provide

- Same data contract as the dashboard stats (or one combined `GET /api/projects/{id}/analytics` returning `avgCycleTime`, `days[]` (completed per day), `distribution[]`, `cycleSeries[]` (per-closed-issue cycle time), `openCount`, `closedCount`, `total`).
- Real implementation should compute from the DB (SQL window functions or service-layer aggregation); the client can keep rendering with recharts.

4.8 Chat (Project-Scoped Realtime Group Chat)

What it does (`/app/chat`) — a Slack/Discord-style channel per project, named `#<project-name-slugified>`:

- Send text messages (Enter to send, Shift+Enter for newline).
- **@mentions**: type `@` → live suggestion list of project members (name/email match, excludes you, max 6); ArrowUp/Down to navigate, Enter to complete, Escape to dismiss; the completed mention renders as a teal chip in the message.
- **Stickers**: a sticker picker with a themed pack (`src/lib/stickers.js`) — e.g. 🚀 "Ship it", ✅ "Approved", 🔍 "Review this", 🔗 "Linked", 🔄 "Merge me", 🐛 "Bug spotted", 🔥 "On fire", ☕ "Coffee break" — rendered as emoji on vibrant gradient tiles. ~18% of simulated replies use stickers.
- **Emoji reactions**: hover a message → quick-reaction picker (👍 🚀 ❤️ etc.); reactions show counts and highlight when you've reacted; toggle on/off.
- **Code blocks**: messages can contain code snippets that auto-format as blocks with a copy affordance (the `RichText` / `CodeBlock` components).
- **Typing indicators**: animated dots + "X, Y typing..." capped at 3 avatars.
- **Presence**: online/offline dots on avatars.
- Message history per project, newest at the bottom, auto-scroll.
- A **chat notification** (and **mention notification**) is created when someone mentions you or replies.

Data shape:

```
// chat: { projectId: [message, ...] }
{ id, projectId, authorId, body, createdAt, sticker?, code? }
// reactions: { messageId: [{ emoji, userId }] }
```

Realtime simulation today (`src/lib/realtime.js`) — this tells you exactly what events a real socket must carry:

1. A random online teammate starts "typing" after 0.9–2.0 s, and clears it after replying.
2. A reply arrives 2.4–4.7 s later: 18% sticker, ~30% of non-sticker messages are @mentions of the current user ("can you take a look when you get a sec?" etc.), ~30% include a code snippet, rest are plain replies.
3. Mentions (and ~30% of replies) generate a notification: type `mention` or `chat`.

Backend must provide

- `GET /api/projects/{id}/messages?before=<id|ts>&limit=50` (paged history), `POST /api/projects/{id}/messages`.
- **Typing events** (throttled; frontend fires at most one per 800 ms): WebSocket `typing.start` / `typing.stop`.
- **Reactions:** `PUT/DELETE /api/messages/{id}/reactions/{emoji}` (idempotent toggle).
- **Mentions:** parse `@Name` mentions server-side; create `mention` notifications for the mentioned users; store mention targets on the message for future "mentioned" views.
- Real-time fan-out of new messages + reactions + typing to all members of the project channel (see §8 and §10).
- Stickers are a client concept (emoji + gradient tile); backend just stores a `sticker` string id. Validation against the known pack is optional.

4.9 AI Project Assistant

What it does (`/app/ai`) — a chat-style assistant **scoped to the active project** that answers from live board data. It is currently **rule-based client-side** (no LLM), but the backend is where a real LLM integration belongs.

- It builds a **project snapshot** from live data:

```
{
  open, done,
  byStatus: { backlog, todo, inProgress, review, done },
  high,      // open issues with priority ≡ 'high'
  stale,     // open issues not updated in > 5 days
  commits,   // total linked commits in the project
  messageCount, // total chat messages
  memberStats, // [{ id, name, assignedOpen }] sorted desc
}
```

- It answers questions about: **status/health**, **team overload & capacity** (top-loaded + lightest member), **blockers/risks** (top high-priority keys + stale count), and **next 48h planning** (move review→done, pull high-priority backlog into todo, resolve stale with owner+deadline).
- Quick-prompt chips, a project snapshot card (open/done/high/stale), and a team-load panel.
- Disclaimer: "Suggestions are advisory."

Backend must provide

- `POST /api/projects/{id}/ai/ask` — receives `{ question }`, returns `{ answer }`. Implementation options:
 - **Recommended:** server-side snapshot + LLM call (OpenAI/Anthropic via Spring AI or plain REST), with the snapshot injected into the system prompt. The snapshot contract above is exactly what the client renders today, so return it alongside the answer.

- Fallback: keep the deterministic rule-based answers server-side first, then swap in an LLM.
 - If issues are created "by AI" (the `aiGenerated` flag exists in the data model), an endpoint like `POST /api/issues/ai-generate` could create issues from natural language later. The UI already knows how to render the `aiGenerated` badge.
-

4.10 Members & Roles

What it does (`/app/members`) — two tabs:

- **Project tab**: members of the active project (with role chips).
- **Workspace tab**: members of the active workspace.
- **Add member** (Owner/Admin only, opens a modal), **change role** (dropdown), **remove member**.
- Non-managers see a banner: *"Only Owner/Admin can manage members — you can view."*

Roles in the code:

- `WORKSPACE_ROLES = ['Owner', 'Admin', 'Member']`
- `PROJECT_ROLES = ['Owner', 'Member']`
- `canManageMembers(scope, id)` returns true iff the current user's role is `Owner` or `Admin` in that scope.

Membership/role actions in the store (replicate server-side):

- `addWorkspaceMember(wsId, userId, role='Member')` — no-op if already a member ("User is already a workspace member").
- `removeWorkspaceMember` — also removes the user from **all projects** of that workspace (and clears roles).
- `setWorkspaceRole(wsId, userId, role)`.
- `addProjectMember(projectId, userId)` — requires prior workspace membership (see §4.3).
- `removeProjectMember`, `setProjectRole`.
- `leaveWorkspace(wsId)` — with **ownership transfer** and **auto-delete of empty workspaces** (see §6).

Backend must provide

- All member CRUD + role endpoints (listed in §4.2/§4.3).
 - **Authorization enforcement** at both the workspace and project scope, matching the matrix in §7.
 - Cascade rules: removing a workspace member removes them from all that workspace's projects; deleting a workspace removes memberships; owner leaving transfers ownership; an empty workspace is deleted.
-

4.11 Invites

What it does — project and workspace invitations shown as a **Tinder-style card stack** immediately after login (`InviteStack`):

- Card shows: workspace chip + role badge (`Owner/Admin/Member · project|workspace`), project/workspace identity (icon, name, description), inviter (avatar + name + "asked you to join this project"/"invited you to this workspace" + relative time), the invitation message, a member-preview avatar stack.

- **Accept** (→) flies the card right with a green glow; **decline** (←) swoops it left with a red glow. Drag gestures, keyboard arrows (←/→), and the two round buttons all work. "NOPE"/"LIKE" stamps appear while dragging.
- Accepting a **project invite**: adds you to the project's `memberIds` + `roles` with the invited role, sets it as the active project, switches to its workspace.
- Accepting a **workspace invite**: adds you to the workspace `memberIds` + `roles`.
- Declining just removes the invite.
- Invite decisions **persist** across refreshes (`invites` is in the persisted part of the store), so a decided invite never reappears.

Seed invite shape:

```
{
  id, type: 'project' | 'workspace',
  workspaceId, projectId (only for type 'project'),
  fromUserId, role, message, createdAt,
}
```

Backend must provide

- `POST /api/workspaces/{id}/invites` and `POST /api/projects/{id}/invites` (inviter must be Owner/Admin of that scope — the seed shows Owners inviting; make it Owner/Admin for parity with member management).
- `GET /api/invites/pending` (for the current user), `POST /api/invites/{id}/accept`, `POST /api/invites/{id}/decline`.
- **Accept must be transactional**: add membership + role, mark invite consumed, set context.
- Realtime: a newly accepted member should appear to other members of that workspace/project (optional; can be covered by "members changed" push).

4.12 Notifications

What it does (`NotificationsPanel` in the top bar):

- Types (with icons): `commit` (git), `mention` (at-sign), `status` (tag), `chat` (message), `invite` (mail).
- Notification shape:

```
{ id, type, title, body, issueId?, projectId?, read: boolean, createdAt }
```

- Unread count badge on the bell (coral pill).
- **Mark all read, clear all, dismiss individual** (hover X), **click to open** → navigates to the linked issue (sets active project first).
- Unread items have a teal tint + dot.
- **Cap**: notifications list is truncated to the **60 most recent** (`[ ... new, ... existing].slice(0, 60)`).

What generates notifications today (must generate server-side):

- `commit` — someone pushes/links a commit to an issue ("Linus pushed to DEV-102").

- `mention` — someone @mentions you in chat or on an issue comment ("Grace mentioned you in DEV-103").
- `status` — someone moves an issue ("DEV-102 moved to In Progress").
- `chat` — someone replies/mentions in the project channel.
- `invite` — someone invites you ("Katherine invited you to Data Platform").

Backend must provide

- `GET /api/notifications?limit=60`, `PATCH /api/notifications/{id}/read`, `POST /api/notifications/read-all`, `DELETE /api/notifications/{id}`, `DELETE /api/notifications`.
 - **Server-side creation** of notifications on: issue status change (for followers/assignee/reporter), commit link, mention, chat mention, invite.
 - Real-time push of new notifications to the recipient (WebSocket or SSE) so the bell badge updates live.
 - Unread count endpoint (`GET /api/notifications/unread-count`).
-

4.13 Global Search (⌘K)

What it does (`SearchModal` , opened with ⌘K / Ctrl+K anywhere in the app, or the search box in the top bar):

- Scoped to the **active workspace only**.
- Searches three entity types simultaneously:
 - **Issues** — matches `key`, `title`, `labels`, `description` (max 6).
 - **Projects** — matches name (max 4).
 - **Members** — matches name/email, workspace members only (max 4).
- Keyboard navigation: ↑/↓ to move, Enter to open, grouped results ("Issues / Projects / Members").
- **Issue result** → opens issue detail (switches active project first). **Project result** → switches project, goes to board. **Member result** → members page.

Backend must provide

- `GET /api/workspaces/{id}/search?q= ... &limit= ...` returning grouped results `{ issues, projects, users }` (or separate endpoints + client-side merging). Should be **case-insensitive substring** matching, workspace-scoped, and permission-filtered.
 - (Later: PostgreSQL full-text search / pg_trgm for performance.)
-

4.14 Settings

What it does (`/app/settings`) — two cards:

1. **Workspace settings** (any member can edit name/description/logo/accent — the frontend allows it, though you may want Owner/Admin-only server-side):
 - Logo picker (a set of `WORKSPACE_LOGOS`), workspace name, **slug** (read-only display, "auto-derived, lowercase"), description, accent color.
2. **Active project integration** (shown when a project is active):

- `repoUrl` — **Owner-only editable** in the UI; members may only edit the local clone path and IDE deep link. The frontend never sends the repo URL for non-owners: `if (isOwner) patch.repoUrl = ...`.
- `localRepoPath`, `ideUrl`.

3. **Danger zone** (Owner only): **Delete workspace** with confirmation modal ("This permanently removes all projects, issues, chat history and members.").

Backend must provide

- `PUT /api/workspaces/{id}` and `PUT /api/projects/{id}` with **role-aware field permissions** (`repoUrl` Owner-only server-side, not just client-side).
- Cascade delete for workspaces; soft-delete consideration.

4.15 Theming

- Two themes: **dark** (graphite surfaces, luminous accents) and **brutal** (soft-brutalist white/ink, hard shadows). Toggled from the landing nav and the top bar. Persisted in `localStorage`.
- All colors are **CSS variables** (design tokens) in `src/index.css` (`--surface-card`, `--signal-teal`, `--text-ink`, ...). Status/priority colors are defined once in `constants.js` and reused everywhere.
- **Backend impact:** none functionally — the client stores the preference. (Backend may store the preference server-side later for cross-device sync; not required.)

4.16 Landing Page (Marketing)

A polished marketing site (not part of the app shell): scroll-driven **pipeline walkthrough** (GSAP pinned horizontal scroll through the 5 stages), feature grid, chat showcase, animated metrics (1.4 days avg close / 142k+ commits linked / 99.99% uptime), and a final CTA. Purely static — no backend needed.

5. Domain Model (Entities, Fields, Relationships)

Below is the canonical entity model **as the frontend already implements it** (from `seed.js` and the store). The backend's JPA entities and DB schema should mirror this exactly. All `id`s are opaque strings (the frontend generates `uid` strings like `ws_x`, `p_x`, `i_x`; the backend can use UUIDs or bigserial — see §12 for the recommendation).

```

User —< WorkspaceMember (workspace.memberIds + roles)
User —< ProjectMember (project.memberIds + roles)
User —< Issue (assignee, reporter)
User —< Comment, Commit (author)
User —< ChatMessage (author)
User —< Reaction
User —< Invite (from), Invite (to)
User —< Notification (to)

Workspace 1< Project 1< Issue 1< Comment 1< Reply
      Issue 1< Commit
      Issue 1< StatusHistoryEntry
      Project 1< ChatMessage 1< Reaction
      Project 1< Invite (project invites)
Workspace 1< Invite (workspace invites)
Workspace 1< Notification (denormalized projectId for routing)

```

5.1 User

Field	Type	Notes
<code>id</code>	string	e.g. <code>u1</code>
<code>name</code>	string	display name
<code>email</code>	string	unique, lowercased on lookup; used for sign-in + mentions
<code>color</code>	string (hex)	avatar color from <code>AVATAR_COLORS</code> (9 options)
<code>online</code>	boolean	presence flag (green dot)
<code>avatar</code>	string (optional)	data URL (uploaded photo ≤ 2 MB) or image URL; empty = initials avatar

5.2 Workspace

Field	Type	Notes
<code>id</code>	string	
<code>name</code>	string	required
<code>slug</code>	string	derived: lowercase, <code>[^a-z0-9]</code> → <code>-</code>
<code>icon</code>	string	logo id from <code>WORKSPACE_LOGOS</code> (hexagon, layers, wave, ...)
<code>accent</code>	string (hex)	from <code>PROJECT_ACCENTS</code>
<code>description</code>	string (optional)	
<code>memberIds</code>	string[]	membership
<code>roles</code>	map userId → role	<code>Owner</code> <code>Admin</code> <code>Member</code>

5.3 Project

Field	Type	Notes
id	string	
workspaceId	string	FK → Workspace
name	string	required
description	string (optional)	
keyPrefix	string	derived: first 3 letters uppercased, fallback PRJ
color	string (hex)	accent from PROJECT_ACCENTS
icon	string	shown as tile glyph (# in seed)
repoUrl	string (optional)	GitHub repo; Owner-editable
localRepoPath	string (optional)	local clone path
ideUrl	string (optional)	deep link override; else derived from localRepoPath
memberIds	string[]	subset of workspace members
roles	map userId → role	Owner Member

5.4 Issue

Field	Type	Notes
id	string	
key	string	PREFIX-NNN ; unique per project , sequential
projectId	string	FK → Project
workspaceId	string	denormalized (set from project at seed time) — keeps queries scoped
title	string	required
description	string (markdown)	
status	enum	backlog todo in_progress review done
priority	enum	high medium low
labels	string[]	from fixed vocabulary
assigneeId	string null	
reporterId	string	
createdAt	ISO timestamp	
updatedAt	ISO timestamp	bumped on every mutation
aiGenerated	boolean	AI badge
comments	Comment[]	threaded

Field	Type	Notes
<code>commits</code>	<code>Commit[]</code>	linked commits
<code>statusHistory</code>	<code>StatusHistoryEntry[]</code>	audit trail

Comment: `{ id, authorId, body, createdAt, replies: [{ id, authorId, body, createdAt }] }`

Commit: `{ id, sha (40 hex), message, branch, authorId, createdAt }`

StatusHistoryEntry: `{ from: status|null, to: status, by: userId, at: ISO }`

5.5 ChatMessage

`{ id, projectId, authorId, body, createdAt, sticker?: string, code?: string }`

- `sticker` — sticker pack id (e.g. `'party'`); body may be empty when a sticker is sent.
- `code` — a raw code snippet string; client renders it as a code block.

5.6 Reaction

Keyed by message: `reactions[messageId] = [{ emoji, userId }]`. Toggle semantics: adding an existing (emoji, user) pair removes it.

5.7 Invite

`{ id, type: 'project'|'workspace', workspaceId, projectId?, fromUserId, role, message, createdAt }`

5.8 Notification

`{ id, type: 'commit'|'mention'|'status'|'chat'|'invite', title, body, issueId?, projectId?, read: boolean, createdAt }` — capped at 60 most recent per user.

6. Business Rules & Invariants

These are **implemented in the frontend store today** — the backend must enforce them server-side (they are the source of truth):

1. **Scoping:** Every piece of data is reachable only within the user's workspace membership. Users see only workspaces they belong to; projects only within those workspaces; issues/chat/analytics only within projects they belong to.
2. **Project membership requires workspace membership.** You cannot add a user to a project unless they are already a member of the project's workspace.
3. **Removing a workspace member** also removes them from every project of that workspace.
4. **Deleting a workspace** cascades to projects → issues (incl. comments/commits/statusHistory), chat, notifications.
5. **Ownership transfer on leave:** when a workspace Owner leaves, ownership is promoted to the first remaining member; if no members remain, the workspace is deleted.

- 6. **Issue key:** `PREFIX-NNN` sequential per project; `NNN` = max existing + 1, zero-padded to 3. Duplicated issues get a new key (and a `(copy)` suffix).
- 7. **Duplicate resets:** `status` → `backlog`, `comments/commits` cleared, new `statusHistory` `[{from:null,to:'backlog'}]`, `aiGenerated` → false.
- 8. **Status transitions append to** `statusHistory` — never overwrite; the timeline is the product.
- 9. **WIP limits:** todo 6, in_progress 5, review 4 (advisory in the UI today).
- 10. **Aging:** > 5 days since `updatedAt` on an open issue = aging (amber); > 8 days = stale (coral).
- 11. **Notifications capped at 60** newest-first per user.
- 12. **Mentions:** `@Name` in chat/comments triggers a notification for the mentioned member and the frontend renders mention chips.
- 13. **Invite consumption:** accept/decline are one-shot; accepted invites add memberships + roles; decisions persist.
- 14. **Email uniqueness** for users (implied by sign-in matching).
- 15. **Validation rules from the UI:** issue title required; workspace/project names required; password ≥ 4 chars; email regex `^\S+@\S+\.\S+$`; profile photo ≤ 2 MB and image/*; name ≤ 40 chars; email ≤ 80 chars.
- 16. **Default member role is** `Member` when added without a role.
- 17. **Role-aware field permissions:** `repoUrl` may only be changed by the project Owner (workspace Owner) — the frontend already enforces this and the backend must too.

7. Roles & Permissions Matrix

Workspace scope

Action	Owner	Admin	Member
View workspace & its projects	✓	✓	✓
Switch workspace	✓	✓	✓
Create project	✓ (all can in UI; store doesn't restrict)	✓	✓
Edit workspace (name/logo/desc/accent)	✓	✓	✓ (UI allows)
Manage members (add/remove/change role)	✓	✓	✗
Send invites	✓ (recommended)	✓ (recommended)	✗
Delete workspace	✓	✗	✗
Leave workspace	✓ (triggers transfer)	✓	✓

Project scope

Action	Owner	Member
View project (board/issues/chat/analytics)	✓	✓
Create/edit/move/delete issues	✓	✓
Comment, link commits, chat	✓	✓
Add/remove project members	✓ (via workspace member pool)	✗
Change project roles	✓	✗
Edit <code>repoUrl</code>	✓	✗ (may edit local path/IDE link)
Delete project	✓ (recommended)	✗

Note for the backend: the frontend is permissive in some UI areas (any member can hit "New Issue"), so the backend's authorization can be slightly stricter than today's UI without breaking anything — but it must never be *more* permissive on member management and deletion than the matrix above.

8. Realtime Requirements

The product is collaborative: multiple members watch the same board, chat channel, and notification bell. The current frontend **simulates** this (`src/lib/realtime.js`); the backend must make it real.

Required realtime channels (per project)

1. **Board / issues channel** — events:

- `issue.created`, `issue.updated` (any field), `issue.moved` (status change, with the new statusHistory entry), `issue.deleted`, `issue.duplicated`.

2. **Chat channel** (per project) — events:

- `chat.message` (new message, with author + sticker/code fields),
- `chat.reaction` (added/removed),
- `chat.typing.start` / `chat.typing.stop` (user id; client throttles to ~1 ping / 800 ms),
- `chat.presence` (member online/offline).

3. **Notification channel** (per user) — event:

- `notification.new` (pushed to the recipient so the bell badge updates instantly).

4. **Membership channel** (per workspace/project, optional) — `members.changed` so member lists and the invite stack stay fresh.

Transport recommendation

- **WebSocket with STOMP** (Spring `spring-boot-starter-websocket` + SockJS fallback) — fits the frontend's event-style needs and is the standard for this stack.
- Destination naming convention:

- Client → server commands: `/app/projects/{projectId}/...`
- Server → clients: `/topic/projects/{projectId}/issues`, `/topic/projects/{projectId}/chat`, `/topic/users/{userId}/notifications`.
- **Authorization:** subscriptions must be authorized (only project members may subscribe to `/topic/projects/{projectId}/*`); the frontend will send the JWT (e.g. via the STOMP `CONNECT` frame header `Authorization`).
- Presence: a presence service (Redis-based or in-memory heartbeat, or Spring `WebSocketHandler` session tracking) feeds `online` flags and `chat.presence`.
- **Fallback note:** if full WebSockets are too much for the first iteration, **SSE** can cover notifications + board events, but chat typing and message fan-out really want a bidirectional socket — plan for WebSocket.

Idempotency & ordering

- Messages created concurrently must not be lost; include `clientId` (the frontend's generated message id) so the server can dedupe retries.
- Reorder within a column: add a `position` field if cross-client ordering persistence is desired.

9. Proposed REST API Surface

A complete, resource-oriented REST API that covers every feature. Base path `/api`. All endpoints require `Authorization: Bearer <JWT>` unless marked public. (URL structure follows the frontend's existing data hierarchy: workspace → project → issue.)

Auth (public)

Method	Path	Purpose
POST	<code>/api/auth/register</code>	<code>{ name, email, password }</code> → 201 + tokens
POST	<code>/api/auth/login</code>	<code>{ email, password }</code> → <code>{ accessToken, refreshToken, user }</code>
POST	<code>/api/auth/refresh</code>	<code>{ refreshToken }</code> → new access token
POST	<code>/api/auth/logout</code>	revoke refresh token
GET	<code>/api/auth/oauth2/{provider}</code>	GitHub / Google SSO (Spring Security OAuth2)

Users

Method	Path	Purpose
GET	<code>/api/users/me</code>	current user profile
PUT	<code>/api/users/me</code>	update name/email/color/online/avatar
POST	<code>/api/users/me/avatar</code>	multipart upload (≤ 2 MB)

Workspaces

Method	Path	Purpose
GET	/api/workspaces	workspaces I belong to
POST	/api/workspaces	create (creator = Owner)
GET	/api/workspaces/{id}	detail
PUT/PATCH	/api/workspaces/{id}	update name/icon/desc/accent
DELETE	/api/workspaces/{id}	Owner-only, cascade
POST	/api/workspaces/{id}/leave	leave; ownership transfer / auto-delete
GET	/api/workspaces/{id}/members	member list with roles
POST	/api/workspaces/{id}/members	{ userId, role } (Owner/Admin)
PATCH	/api/workspaces/{id}/members/{userId}/role	change role (Owner/Admin)
DELETE	/api/workspaces/{id}/members/{userId}	remove (cascade to projects)
GET	/api/workspaces/{id}/projects	projects in workspace I can see
GET	/api/workspaces/{id}/search	?q= grouped search (issues/projects/users)
GET	/api/workspaces/{id}/invites	sent invites (management)
POST	/api/workspaces/{id}/invites	create workspace invite (Owner/Admin)

Projects

Method	Path	Purpose
POST	/api/workspaces/{id}/projects	create project
GET	/api/projects/{id}	detail
PUT/PATCH	/api/projects/{id}	update (role-aware fields)
DELETE	/api/projects/{id}	delete (Owner)
GET	/api/projects/{id}/members	members
POST	/api/projects/{id}/members	add (workspace-member prerequisite)
PATCH	/api/projects/{id}/members/{userId}/role	change role
DELETE	/api/projects/{id}/members/{userId}	remove
POST	/api/projects/{id}/invites	create project invite (Owner/Admin)
GET	/api/projects/{id}/issues	? status=&priority=&assigneeId=&search=
POST	/api/projects/{id}/issues	create issue (auto key)
GET	/api/projects/{id}/messages	chat history ?before=&limit=

Method	Path	Purpose
POST	<code>/api/projects/{id}/messages</code>	send message
GET	<code>/api/projects/{id}/stats</code>	dashboard aggregates
GET	<code>/api/projects/{id}/analytics</code>	analytics aggregates
GET	<code>/api/projects/{id}/activity</code>	<code>?limit=</code> recent status moves
POST	<code>/api/projects/{id}/ai/ask</code>	<code>{ question }</code> → <code>{ answer, snapshot }</code>

Issues

Method	Path	Purpose
GET	<code>/api/issues/{id}</code>	detail (with comments/commits/statusHistory)
PUT/PATCH	<code>/api/issues/{id}</code>	update fields (appends statusHistory on status change)
DELETE	<code>/api/issues/{id}</code>	delete
POST	<code>/api/issues/{id}/duplicate</code>	duplicate (new key, reset to backlog)
POST	<code>/api/issues/{id}/move</code>	<code>{ to: status }</code> — audit + notify + realtime
POST	<code>/api/issues/{id}/comments</code>	add comment
POST	<code>/api/comments/{id}/replies</code>	reply to comment
POST	<code>/api/issues/{id}/commits</code>	link commit <code>{ sha, message, branch }</code>

Invites

Method	Path	Purpose
GET	<code>/api/invites/pending</code>	invites for current user
POST	<code>/api/invites/{id}/accept</code>	accept (transactional membership)
POST	<code>/api/invites/{id}/decline</code>	decline

Notifications

Method	Path	Purpose
GET	<code>/api/notifications</code>	newest 60
GET	<code>/api/notifications/unread-count</code>	badge count
PATCH	<code>/api/notifications/{id}/read</code>	mark read
POST	<code>/api/notifications/read-all</code>	mark all read
DELETE	<code>/api/notifications/{id}</code>	dismiss
DELETE	<code>/api/notifications</code>	clear all

Chats (message-level)

Method	Path	Purpose
PUT	/api/messages/{id}/reactions/{emoji}	add reaction (idempotent)
DELETE	/api/messages/{id}/reactions/{emoji}	remove reaction

10. Proposed WebSocket Event Contracts

Payload examples (server → client). JSON, sent over STOMP destinations from §8.

```
// /topic/projects/{projectId}/issues
{ "type": "issue.moved",
  "issue": { "id": "i4", "key": "DEV-102", "status": "review",
    "updatedAt": "2026-08-08T10:00:00Z", "statusHistory": [/* latest entry */] } }

// /topic/projects/{projectId}/chat
{ "type": "chat.message",
  "message": { "id": "m98", "projectId": "p1", "authorId": "u2",
    "body": "@Ada can you sanity-check the diff?", "createdAt": "..." } }
{ "type": "chat.typing.start", "projectId": "p1", "userId": "u2" }
{ "type": "chat.typing.stop", "projectId": "p1", "userId": "u2" }
{ "type": "chat.reaction", "messageId": "m98", "emoji": "👍",
  "userId": "u1", "added": true }

// /topic/users/{userId}/notifications
{ "type": "notification.new",
  "notification": { "id": "n120", "type": "mention", "title": "...", "body": "...",
    "issueId": "i5", "projectId": "p1", "read": false, "createdAt": "..." } }

// /topic/projects/{projectId}/presence
{ "type": "chat.presence", "userId": "u2", "online": true }
```

Client → server commands (destinations `/app/ ...`):

```
// /app/projects/{projectId}/typing
{ "userId": "u1", "projectId": "p1" }           // start; client throttles to 1/800ms
// /app/projects/{projectId}/messages/read      // optional read-receipts (not in current UI)
```

Server responsibilities on these events: persist first, fan out second; reject events for non-member subscriptions; create notifications for mentions/commits/status changes.

11. Recommended Backend Architecture (Spring Boot)

11.1 Stack (recommended baseline)

Concern	Choice	Why
Language / runtime	Java 17 or 21	LTS, records/sealed types, virtual threads (21)
Framework	Spring Boot 3.3.x+	Current stable; Jakarta EE 10
Build	Maven (or Gradle)	Standard
REST	Spring Web (<code>spring-boot-starter-web</code>)	
Persistence	Spring Data JPA + Hibernate	Mature ORM
Database	PostgreSQL 16	Relational, JSONB for label arrays if wanted
Migrations	Flyway	Versioned DDL
Security	Spring Security 6 + JWT (jjwt or nimbus)	Stateless API auth
OAuth2	Spring Security OAuth2 Client	GitHub/Google SSO
Validation	<code>spring-boot-starter-validation</code> (Bean Validation)	Match UI rules (§6)
Realtime	<code>spring-boot-starter-websocket</code> (STOMP + SockJS)	§8/§10
Docs	springdoc-openapi (Swagger UI)	API docs
Boilerplate	Lombok (or records for DTOs)	Less code
Caching/rate-limit (optional)	Redis + <code>spring-boot-starter-data-redis</code>	Presence, token-bucket rate limiting, session store
Testing	JUnit 5, Mockito, MockMvc, Testcontainers	§16
Observability	Actuator + Micrometer	Health, metrics

11.2 Package layout (recommended)

```
com.devflow
├── DevFlowApplication.java
├── config/
│   ├── SecurityConfig.java           # JWT filter chain, OAuth2, CORS
│   ├── WebSocketConfig.java          # STOMP endpoints, auth interceptor
│   ├── OpenApiConfig.java
│   └── AsyncConfig.java              # @Async for notifications
├── security/
│   ├── JwtService.java               # issue/verify access+refresh tokens
│   ├── JwtAuthFilter.java            # OncePerRequestFilter → SecurityContext
│   └── CurrentUser.java              # accessor for authenticated principal
├── user/
│   ├── User.java   UserRepository.java   UserService.java   UserController.java
│   └── dto/ (UserDto, UpdateUserRequest, AvatarUploadResponse...)
├── workspace/
│   ├── Workspace.java   WorkspaceRepository.java   WorkspaceService.java
│   ├── WorkspaceController.java
│   └── member/ (WorkspaceMember embedded, role enum, DTOs)
├── project/
│   ├── Project.java   ProjectRepository.java   ProjectService.java   ProjectController.java
│   └── member/ (ProjectMember, ProjectRole)
├── issue/
│   ├── Issue.java   IssueRepository.java   IssueService.java   IssueController.java
│   ├── Status.java   Priority.java   Label.java (enums)
│   ├── Comment.java   Reply.java   LinkedCommit.java   StatusHistoryEntry.java
│   └── dto/ (IssueDto, CreateIssueRequest, MoveIssueRequest...)
├── chat/
│   ├── ChatMessage.java   ChatMessageRepository.java   ChatService.java
│   ├── Reaction.java   TypingController.java (WS command handler)
│   └── dto/
├── invite/
│   ├── Invite.java   InviteRepository.java   InviteService.java   InviteController.java
│   └── dto/
├── notification/
│   ├── Notification.java   NotificationRepository.java   NotificationService.java
│   └── dto/
├── analytics/
│   ├── AnalyticsService.java          # aggregates (cycle time, throughput, distribution)
│   └── AiAssistantService.java         # snapshot + LLM / rule-based answers
├── realtime/
│   ├── WsEventPublisher.java          # SimpMessagingTemplate wrappers
│   └── PresenceService.java
└── common/
    ├── GlobalExceptionHandler.java    # @RestControllerAdvice
    ├── ApiError.java   PageResponse.java   UidGenerator.java
    └── audit/ (CreatedAt, UpdatedAt base entity, @PrePersist/@PreUpdate)
```

11.3 Architecture principles

- **Layered:** Controller → Service → Repository; DTOs in/out (never expose entities directly).
- **Centralized authz:** a `WorkspaceAccessService` / `ProjectAccessService` helper
(`requireWorkspaceMember(userId, wsId)` , `requireWorkspaceOwnerOrAdmin` , `requireProjectMember` , `requireProjectOwner`) called at the top of every service method. *Do not scatter raw role checks.*

- **Transactional mutations:** every write that touches multiple aggregates (accept invite, delete workspace cascade, add project member, status move + audit + notification) runs in `@Transactional`.
 - **Audit on the entity:** base entity with `createdAt` / `updatedAt` via `@PrePersist` / `@PreUpdate` (or Hibernate `@CreationTimestamp` / `@UpdateTimestamp`).
 - **Optimistic locking** on Issue (`@Version`) to avoid lost updates on concurrent status changes.
 - **Pagination** everywhere lists can grow (messages, notifications, issues, search).
 - **Uniform error responses** via `@RestControllerAdvice` (`{ status, code, message, fieldErrors? }`), matching the toasts the frontend shows (success/error/info).
 - **Async side effects:** notification creation and WebSocket fan-out after persistence via `@Async` or `ApplicationEventPublisher` (Spring events) to keep request latency low.
 - **Realtime:** use Spring events (`NotificationCreatedEvent` , `IssueMovedEvent` , `ChatMessageCreatedEvent`) consumed by a realtime listener that publishes to STOMP — keeps domain code decoupled from the socket layer.
-

12. Database Schema (PostgreSQL DDL)

Recommended schema (Flyway `v1__init.sql`). Primary keys: use **UUID** (`gen_random_uuid()`) to match the frontend's string-id worldview, or `BIGSERIAL` if you prefer. The frontend will need to switch from generating its own ids to using server ids — UUIDs make that seamless.

```

-- ===== USERS =====
CREATE TABLE app_user (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(40)  NOT NULL,
  email       VARCHAR(80)  NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,          -- BCrypt
  color       VARCHAR(9)   NOT NULL DEFAULT '#009E88',
  online      BOOLEAN      NOT NULL DEFAULT FALSE,
  avatar_url  TEXT,          -- stored upload URL or data URL
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- ===== WORKSPACES =====
CREATE TABLE workspace (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(120) NOT NULL,
  slug        VARCHAR(140) NOT NULL UNIQUE,
  icon        VARCHAR(32)  NOT NULL DEFAULT 'hexagon',
  accent      VARCHAR(9)   NOT NULL DEFAULT '#009E88',
  description TEXT,
  created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TYPE workspace_role AS ENUM ('Owner', 'Admin', 'Member');

CREATE TABLE workspace_member (
  workspace_id UUID NOT NULL REFERENCES workspace(id) ON DELETE CASCADE,
  user_id      UUID NOT NULL REFERENCES app_user(id)  ON DELETE CASCADE,
  role         workspace_role NOT NULL DEFAULT 'Member',
  joined_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (workspace_id, user_id)
);

-- ===== PROJECTS =====
CREATE TABLE project (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID NOT NULL REFERENCES workspace(id) ON DELETE CASCADE,
  name        VARCHAR(120) NOT NULL,
  description  TEXT,
  key_prefix   VARCHAR(6)   NOT NULL,          -- 'DEV', 'API'...
  color       VARCHAR(9)   NOT NULL DEFAULT '#009E88',
  icon        VARCHAR(8)   NOT NULL DEFAULT '#',
  repo_url     TEXT,
  local_repo_path TEXT,
  ide_url      TEXT,
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
  CONSTRAINT uq_project_prefix_in_ws UNIQUE (workspace_id, key_prefix) -- optional
);

CREATE INDEX idx_project_workspace ON project(workspace_id);

CREATE TYPE project_role AS ENUM ('Owner', 'Member');

CREATE TABLE project_member (
  project_id UUID NOT NULL REFERENCES project(id)  ON DELETE CASCADE,
  user_id    UUID NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
  role       project_role NOT NULL DEFAULT 'Member',

```

```

        joined_at TIMESTAMPTZ NOT NULL DEFAULT now(),
        PRIMARY KEY (project_id, user_id)
    );

-- ===== ISSUES =====
CREATE TYPE issue_status AS ENUM ('backlog','todo','in_progress','review','done');
CREATE TYPE issue_priority AS ENUM ('high','medium','low');

CREATE TABLE issue (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id  UUID NOT NULL REFERENCES project(id) ON DELETE CASCADE,
    workspace_id UUID NOT NULL REFERENCES workspace(id) ON DELETE CASCADE, -- denormalized for
scoping
    key         VARCHAR(16) NOT NULL,          -- 'DEV-001'
    title       VARCHAR(300) NOT NULL,
    description  TEXT,
    status      issue_status NOT NULL DEFAULT 'backlog',
    priority    issue_priority NOT NULL DEFAULT 'medium',
    labels      TEXT[] NOT NULL DEFAULT '{}', -- fixed vocabulary; validated in
service
    assignee_id UUID REFERENCES app_user(id) ON DELETE SET NULL,
    reporter_id UUID NOT NULL REFERENCES app_user(id),
    ai_generated BOOLEAN NOT NULL DEFAULT FALSE,
    position    INTEGER,                      -- optional: column ordering
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    version     INTEGER NOT NULL DEFAULT 0,    -- @Version optimistic lock
    CONSTRAINT uq_issue_key UNIQUE (project_id, key)
);

CREATE INDEX idx_issue_project      ON issue(project_id);
CREATE INDEX idx_issue_workspace   ON issue(workspace_id);
CREATE INDEX idx_issue_status      ON issue(status);
CREATE INDEX idx_issue_assignee    ON issue(assignee_id);

CREATE TABLE status_history (
    id          BIGSERIAL PRIMARY KEY,
    issue_id    UUID NOT NULL REFERENCES issue(id) ON DELETE CASCADE,
    from_status issue_status,
    to_status   issue_status NOT NULL,
    changed_by  UUID NOT NULL REFERENCES app_user(id),
    changed_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_status_history_issue ON status_history(issue_id, changed_at);

CREATE TABLE issue_comment (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    issue_id    UUID NOT NULL REFERENCES issue(id) ON DELETE CASCADE,
    author_id   UUID NOT NULL REFERENCES app_user(id),
    body        TEXT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE comment_reply (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    comment_id  UUID NOT NULL REFERENCES issue_comment(id) ON DELETE CASCADE,
    author_id   UUID NOT NULL REFERENCES app_user(id),
    body        TEXT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE TABLE linked_commit (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  issue_id    UUID NOT NULL REFERENCES issue(id) ON DELETE CASCADE,
  sha         CHAR(40) NOT NULL,
  message     TEXT NOT NULL,
  branch      VARCHAR(200) NOT NULL DEFAULT 'main',
  author_id   UUID NOT NULL REFERENCES app_user(id),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ===== CHAT =====
CREATE TABLE chat_message (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  UUID NOT NULL REFERENCES project(id) ON DELETE CASCADE,
  author_id   UUID NOT NULL REFERENCES app_user(id),
  body        TEXT NOT NULL DEFAULT '',
  sticker     VARCHAR(32),                -- sticker pack id
  code        TEXT,                       -- code snippet payload
  client_id   VARCHAR(64),                -- dedupe client-generated ids
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_msg_client UNIQUE (project_id, client_id)
);
CREATE INDEX idx_msg_project ON chat_message(project_id, created_at);

CREATE TABLE reaction (
  message_id  UUID NOT NULL REFERENCES chat_message(id) ON DELETE CASCADE,
  user_id     UUID NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
  emoji       VARCHAR(16) NOT NULL,
  PRIMARY KEY (message_id, user_id, emoji)
);

-- ===== INVITES =====
CREATE TABLE invite (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  type        VARCHAR(16) NOT NULL CHECK (type IN ('project', 'workspace')),
  workspace_id UUID NOT NULL REFERENCES workspace(id) ON DELETE CASCADE,
  project_id  UUID REFERENCES project(id) ON DELETE CASCADE, -- null for workspace invites
  from_user_id UUID NOT NULL REFERENCES app_user(id),
  to_email    VARCHAR(80),                -- or to_user_id; see note below
  to_user_id  UUID REFERENCES app_user(id),
  role        VARCHAR(16) NOT NULL DEFAULT 'Member',
  message     TEXT,
  status      VARCHAR(16) NOT NULL DEFAULT 'pending' CHECK (status IN
('pending', 'accepted', 'declined')),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  decided_at  TIMESTAMPTZ
);
CREATE INDEX idx_invite_to ON invite(to_user_id, status);

-- ===== NOTIFICATIONS =====
CREATE TABLE notification (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
  type        VARCHAR(16) NOT NULL CHECK (type IN
('commit', 'mention', 'status', 'chat', 'invite')),
  title       VARCHAR(300) NOT NULL,
  body        TEXT NOT NULL,
  issue_id    UUID REFERENCES issue(id) ON DELETE CASCADE,

```

```

project_id UUID REFERENCES project(id) ON DELETE CASCADE,
is_read    BOOLEAN NOT NULL DEFAULT FALSE,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_notification_user ON notification(user_id, created_at DESC);

```

Invites note: the current demo invites target user ids (they're seeded for the demo user). For a real product, invites should target **email addresses** (invite anyone, member or not) and become actionable when that email registers. Design `invite.to_email` accordingly and keep `to_user_id` for instant matches. Accept/decline must be **one transaction**: update invite status + insert memberships + emit realtime.

13. Security Design

1. Stateless JWT:

- Access token: short-lived (15–30 min), contains `sub` (user id), `email`, `roles` -free (roles are per-workspace, resolved from DB on each request).
- Refresh token: long-lived (7–30 days), stored hashed in DB (or Redis) to support revocation/logout.
- `JwtAuthFilter` (OncePerRequestFilter) validates signature + expiry and populates `SecurityContext` with a `PrincipalUser`.

2. Password storage: BCrypt (`BCryptPasswordEncoder` , strength 10+).

3. Authorization model: fine-grained, data-driven (per workspace/project membership + role), via the access-service helpers (§11.3). Spring's `@PreAuthorize` is best for coarse rules; the membership checks must be method-level DB queries.

4. WebSocket auth: STOMP `CONNECT` must include `Authorization: Bearer <jwt>`; an interceptor validates it, and subscriptions to `/topic/projects/{id}/*` are checked against project membership.

5. CORS: allow the Vite dev origin (and later the deployed frontend); credentials not needed if using bearer tokens.

6. Input validation: Bean Validation matching UI rules (§6); labels validated against the fixed vocabulary; issue keys generated server-side (never trusted from the client).

7. Rate limiting (optional but recommended): token-bucket per user/IP on auth + message-send endpoints (the seed even contains a "Rate limiting middleware" issue `API-201` — the product's own roadmap expects it).

8. Idempotency: accept `Idempotency-Key` headers on POSTs that create (issues, messages, invites) — the seed references idempotent retries (`API-204`) too.

9. Audit: `status_history` is the domain audit; add request logging (logback) + optional `@Audited` on sensitive mutations.

10. Secrets: JWT secret, DB creds, LLM API keys in environment variables / Spring profiles (never in code).

14. Skills Required to Build This Backend

Must-have (core)

- **Java** (17/21) — language fundamentals, streams, records, exceptions.
- **Spring Boot 3** — autoconfiguration, starters, profiles, `application.yml`, Actuator.
- **Spring Web / REST** — controllers, `@RequestBody` / `@PathVariable`, DTO mapping, status codes, error handling.
- **Spring Data JPA / Hibernate** — entities, repositories, derived queries, `@Query`, `@Transactional`, entity relationships, `@Version`, auditing.
- **Spring Security 6** — filter chains, stateless JWT auth, method security, CORS, password encoding.
- **SQL / PostgreSQL** — schema design, indexes, transactions, `TIMESTAMPZ`, arrays, window functions (for analytics), `gen_random_uuid()`.
- **Flyway** — versioned migrations.
- **Maven** (or Gradle) — dependencies, profiles, multi-module if needed.
- **Testing** — JUnit 5, Mockito, `@WebMvcTest` / `@DataJpaTest` / `@SpringBootTest`, MockMvc.
- **Git** — branching, PR workflow.

Strongly recommended

- **WebSockets + STOMP** (Spring Messaging) — the realtime chat/board/notification layer. (Or SSE + long-polling as a fallback, but plan for WS.)
- **OAuth2** (Spring Security OAuth2 Client) — GitHub/Google SSO.
- **Bean Validation** — request DTO validation.
- **SpringDoc OpenAPI** — generating Swagger docs for the API.
- **Docker / Docker Compose** — local Postgres, Redis; **Testcontainers** for integration tests.
- **LLM integration** (Spring AI or plain HTTP) — for the AI assistant endpoint.

Nice-to-have / later

- **Redis** — presence, rate limiting, token storage, caching.
- **Flyway + H2** for fast local tests; **Liquibase** alternative.
- **CI/CD** (GitHub Actions: build → test → deploy), containerized deploy (Render/Railway/Fly.io or a VPS).
- Observability: Micrometer/Prometheus/Grafana (the product's own roadmap references a "Chat latency dashboard" `OPS-303`).
- **HTTPS + reverse proxy** (Caddy/nginx) for production WebSockets.
- GraphQL or a BFF later (not needed now — REST is fine).

Effort framing for planning: Core CRUD + auth ≈ 1–2 weeks for one developer; + realtime chat/board events ≈ +1 week; + invites/notifications ≈ +1 week; + analytics/AI endpoints ≈ +3–5 days; hardening (rate limits, idempotency, tests, CI) ongoing. These are rough, context-dependent numbers.

15. Suggested Implementation Phases

Phase 0 — Foundations (day 1–3)

- Scaffold Spring Boot 3 project (Spring Initializr): Web, Data JPA, Security, Validation, WebSocket, PostgreSQL driver, Flyway, Lombok, springdoc.
- Docker Compose for Postgres; `application.yml` with profiles (`dev` , `test`).
- Flyway `v1` schema (§12), entities + repositories for User/Workspace/Project.
- Global exception handler, DTO conventions, base entity auditing.

Phase 1 — Auth & users

- Register/login/refresh/logout, JWT filter, BCrypt.
- `/api/users/me` + avatar upload.
- (Stretch) OAuth2 GitHub/Google.

Phase 2 — Workspaces & projects

- CRUD + slug generation, membership endpoints, role changes, leave with ownership transfer + empty-workspace deletion, cascade delete.
- Scoping helpers (`WorkspaceAccessService` , `ProjectAccessService`) — used by everything after this.

Phase 3 — Issues core

- Issue CRUD, per-project sequential keys (race-safe), status transitions with `statusHistory` append, duplicate, comments/replies, linked commits.
- Board read endpoints with filters.
- Optimistic locking; unit + integration tests.

Phase 4 — Realtime layer

- STOMP endpoints, JWT interceptor, subscription authz.
- Publish `issue.*` events on mutations; presence service.
- (Wire a realtime client later — the current frontend simulates this.)

Phase 5 — Chat

- Message CRUD + pagination, reactions (toggle), typing events (throttled), sticker field, mention parsing → notifications.
- Fan-out over WebSocket.

Phase 6 — Invites & notifications

- Invite CRUD, transactional accept/decline, pending list.
- Notification generation for status/commit/mention/chat/invite; read/unread/clear; unread-count; realtime `notification.new` .
- Notifications capped at 60.

Phase 7 — Analytics & AI

- `stats` / `analytics` aggregates (cycle time, throughput by day, distribution, cycle trend) — SQL or service-layer.
- `/ai/ask` : snapshot + (rule-based first, LLM later) answers.

- `activity` feed endpoint.

Phase 8 — Frontend integration & hardening

- Replace the store's mock layer with an API client (fetch/axios + a small service layer + zustand caching), keeping the same shapes so the UI barely changes.
- Replace `realtime.js` simulations with a STOMP client.
- Rate limiting, idempotency, pagination polish, full test suite, CI.

16. Testing Strategy

- **Unit:** services with Mockito (key generation, ownership transfer, cascade rules, duplicate semantics, invite accept transaction).
- **Slice tests:** `@WebMvcTest` for controllers, `@DataJpaTest` for repositories (unique key constraint, ordering).
- **Integration:** `@SpringBootTest` + **Testcontainers-PostgreSQL** — full flows: register → create workspace → invite → accept → issue lifecycle → status move → notification created.
- **Security:** `@WithMockUser` -style security tests proving the permission matrix (§7) — e.g. a `Member` cannot add members, non-members get 403/404 on scoped endpoints, a removed member loses access immediately.
- **Realtime:** MockMvc + `SimpMessagingTemplate` tests that a status move publishes the expected STOMP payload; Testcontainers Redis if presence is Redis-backed.
- **Analytics:** seed a fixed dataset, assert cycle-time/throughput numbers match the frontend formulas (§4.6).
- **Load/perf (later):** concurrent issue-key generation test (no duplicate keys under 50 parallel creates).

17. Glossary

Term	Meaning
Workspace	Top-level tenant container; owns projects, members, roles (Owner/Admin/Member)
Project	Work stream inside a workspace; own members, issue-key prefix, chat channel, analytics
Issue	Ticket that moves through the 5-stage pipeline; has priority, labels, assignee, comments, commits, statusHistory
Pipeline	The 5 statuses: backlog → todo → in_progress → review → done
Key	Human id <code>PREFIX-NNN</code> , sequential per project
statusHistory	Audit trail of status transitions per issue
Cycle time	Days from creation to Done (from statusHistory)
Throughput	Issues completed per day
WIP limit	Advisory column cap (todo 6, in_progress 5, review 4)
Aging / stale	Open issue untouched 5 / 8 days

Term	Meaning
Sticker	Emoji-on-gradient-tile chat item (pack in <code>src/lib/stickers.js</code>)
Mock / demo layer	The frontend's simulated auth + realtime + seed data, to be replaced by the backend

Appendix A — How to Validate the Backend Against the Frontend

The fastest way to know the backend is faithful: **run the frontend** (`npm install && npm run dev`), and confirm the backend data shapes match what the pages render. The store is the spec — `src/store/useAppStore.js` documents every mutation and business rule; `src/data/seed.js` documents every entity; `src/lib/constants.js` documents every enum. When in doubt, grep those three files.

Appendix B — Immediate Next Step for Claude

If you paste this document into Claude and ask "build the backend", the ideal first deliverable is **Phase 0 + Phase 1 + Phase 3** as a runnable Spring Boot skeleton: schema, entities, auth, workspace/project/issue CRUD with the business rules above, plus a Postgres compose file and a seed-data migration mirroring `src/data/seed.js`. Everything in this document exists so that build can be done without asking further questions about product behavior.